Pemanfaatan Bash Script untuk Penandatanganan Dokumen Digital di Sistem Operasi Linux

Nayla Innocentia Ekatri[1,*], Bima Mukhlisin Bil Sajjad[1], Dafa Novran Hasjrullah[1], Muhamad Mirza Sabil [1], Azura Rizky Annisa [1]
[1]Departemen Teknik Elektro dan Informatika, Sekolah Vokasi, Universitas Gadjah Mada;
[*]Korespondensi: naylainnocentiaekatri@mail.ugm.ac.id;

Abstract - The increasing demand for secure and verifiable electronic documents has driven the need for effective digital signature systems. In Indonesia, digital signatures are legally recognized under the Information and Electronic Transactions Act (UU ITE) and related government regulations. However, technical implementation often remains a challenge for users in practical environments. This article presents the design and implementation of a Bash-based automation script for generating and applying RSA-based digital signatures on PDF files within Debian-based Linux systems. The script performs a sequence of operations, including operating system verification, license agreement validation, RSA certificate generation, and PDF file signing using OpenSSL and OpenPSFSign tools. User interaction is managed through a terminal-based interface that facilitates input sanitization, certificate configuration, and secure file handling. The system ensures the authenticity and integrity of digital documents while simplifying the signing process through automation. The results demonstrate that the tool successfully creates valid digital certificates and applies them to PDF files, providing a lightweight and efficient solution for secure document management. This work contributes to the broader adoption of digital signature practices in open-source environments and supports the legal and technical standards required for digital authentication in Indonesia.

Keywords – Digital Signature, RSA (Rivest-Shamir-Adleman), Bash Scripting, PDF Authentication, Open-Source Automation

Intisari – Kebutuhan terhadap dokumen elektronik yang aman dan dapat diverifikasi terus meningkat, mendorong pentingnya sistem tanda tangan digital yang efektif. Di Indonesia, tanda tangan digital telah diakui secara hukum melalui Undang-Undang Informasi dan Transaksi Elektronik (UU ITE) serta peraturan pemerintah yang berlaku. Namun, implementasi teknisnya masih menjadi tantangan bagi sebagian pengguna. Penelitian ini menyajikan perancangan dan implementasi skrip otomatisasi berbasis Bash untuk menghasilkan dan menerapkan tanda tangan digital berbasis RSA pada file PDF di sistem operasi Linux berbasis Debian. Skrip ini melibatkan beberapa tahap, termasuk verifikasi sistem operasi, validasi persetujuan lisensi, pembuatan sertifikat RSA, serta proses penandatanganan file PDF menggunakan alat OpenSSL dan OpenPSFSign. Antarmuka terminal digunakan untuk mengelola interaksi pengguna, termasuk sanitasi input, konfigurasi sertifikat, dan penanganan file secara aman. Sistem ini memastikan keaslian dan integritas dokumen digital sekaligus menyederhanakan proses penandatanganan melalui otomasi. Hasil implementasi menunjukkan bahwa sistem mampu menghasilkan sertifikat digital yang sah dan berhasil menerapkannya pada file PDF sehingga memberikan solusi yang efisien untuk pengelolaan dokumen digital yang aman. Karya ini berkontribusi terhadap adopsi lebih luas praktik tanda tangan digital di lingkungan *open-source* serta mendukung standar hukum dan teknis yang diperlukan untuk autentikasi digital di Indonesia.

Kata kunci – Tanda Tangan Digital, RSA (Rivest-Shamir-Adleman), Skrip Bash, Otentikasi PDF, Otomatisasi *Open-Source*

PENDAHULUAN

Perkembangan teknologi informasi yang kian cepat mendorong digitalisasi dokumen, termasuk penggunaan tanda tangan digital sebagai pengganti dari tanda tangan konvensional. Tanda tangan digital merupakan suatu teknologi yang digunakan untuk membuktikan bahwa suatu dokumen digital tidak mengalami modifikasi secara ilegal [1]. Penggunaan tanda tangan digital sebagai pengganti dari tanda tangan konvensional memiliki berbagai keunggulan, di antaranya adalah menjamin keaslian dokumen digital dan meningkatkan efisiensi secara keseluruhan. Tanda tangan digital dapat menjadi solusi dalam mengatasi kemungkinan adanya pemalsuan dokumen. Pemanfaatan tanda tangan digital untuk dokumen perlu diadakan supaya suatu dokumen tersebut dapat terjamin otentitasnya dan memperkecil kemungkinan akan adanya pemalsuan dokumen digital.

Konsep tanda tangan digital *(digital signature)* merupakan penerapan kriptografi modern yang berfungsi memastikan keaslian suatu dokumen dan bersifat *non-repudiation* [2]. Tanda tangan digital memiliki peran yang krusial khususnya untuk mempercepat dan mengamankan pertukanan informasi dalam beberapa sektor, seperti keuangan, kesehatan, dan pemerintahan [3]. keamanan dalam proses penandatanganan dokumen digital sangat bergantung pada pengelolaan kunci kriptografi yang digunakan. Penggunaan kunci privat yang aman dan terlindungi adalah aspek krusial agar tanda tangan digital tidak dapat disalahgunakan atau diretas oleh pihak yang tidak berwenang.

Sistem manajemen kunci harus dilakukan secara ketat, termasuk pengamanan kunci privat dan penyimpanan yang aman, serta penerapan protokol distribusi kunci yang terpercaya. Di Linux, Bash script dapat diperkaya dengan mekanisme enkripsi dan pengelolaan kunci yang aman, sehingga proses tanda tangan menjadi lebih andal dan tahan terhadap serangan.

Bash script pada Linux menyediakan solusi yang fleksibel untuk melakukan proses otomatisasi penandatangan dokumen digital. Dengan menggunakan *bash script*, pengguna dapat menyederhanakan proses penandatanganan, serta menjamin konsistensi dan keamanan tanda tangan digital [4]. Otomatisasi ini tidak hanya meningkatkan efisiensi, tetapi juga dapat meminimalkan risiko keselahan manusia sehingga dapat meningkatkan keandalan tanda tangan digital. Efektivitas penggunaan Bash script juga dipengaruhi oleh faktor kompatibilitas dan kemudahan integrasi dengan sistem lain. Linux sebagai sistem operasi open-source memiliki banyak alat dan pustaka yang mendukung operasi kriptografi, seperti OpenSSL dan GPG (GnuPG) yang dapat diintegrasikan langsung ke dalam skrip Bash. Dengan demikian, pengguna tidak perlu mengandalkan perangkat lunak pihak ketiga yang kompleks karena Bash script dapat memanfaatkan pustaka tersebut secara langsung. Pendekatan ini meningkatkan fleksibilitas, mempercepat pengembangan solusi otomatisasi, serta menurunkan biaya implementasi dalam skala organisasi.

Penerapan tanda tangan digital melalui *bash script* turut mendukung SDG *(Sustainable Development Goals)* ke-16, yaitu mengenai perdamaian, keadilan, dan kelembagaan yang kuat. SDG 16 menekankan pentingnya membangun institusi yang efektif, bertanggung jawab, dan transapan [5]. Tanda tangan digital mendukung tujuan ini dalam menyediakan metode autentikasi dokumen yang dapat diverifikasi dan memastikan bahwa setiap dokumen yang ditandatangani dapat dilacak ke pembuatnya. Hal ini akan meningkatkan transparasi dalam sebuah organasasi maupun lembaga.

DASAR TEORI

a. Bash Script

Bash (*Bourne Again Shell*) merupakan salah satu antarmuka yang digunakan oleh *developer* untuk berinteraksi dengan sistem operasi Linux. Pengguna dapat membuat program yang ditulis dalam bentuk file teks biasa atau dikenal sebagai *Bash Scripting*. Isi dari *Sript Bash* ini dapat mencakup perintah internal Bash, perintah GNU/Linux atau Unix, aplikasi lain, maupun skrip tambahan [6]. Bash telah menjadi salah satu shell utama di berbagai distribusi GNU/Linux sejak akhir 1998. Bash Shell dirancang khusus untuk mengelola tugas-tugas sistem dan pengalokasian sumber daya di lingkungan berbasis Linux. Bash menawarkan berbagai utilitas fleksibel melalui sintaks dan struktur perintahnya, termasuk kemampuan seperti ekspansi wildcard, pencocokan pola, pernyataan *conditional control*, *pipe redirection*, substitusi perintah, dan manipulasi nama file [7].

b. OpenSSL

OpenSSL merupakan *open-source project* yang dirancang untuk mendukung penerapan protokol TLS (*Tranport Layer Security*), yaitu protokol yang digunakan untuk menjaga keamanan komunikasi data di jaringan [8]. Setelah dikembangkan selama lebih dari 25 tahun, OpenSSL telah tumbuh menjadi salah satu pustaka (*library*) kriptografi paling populer di dunia. Banyak aplikasi dan sistem menggunakan OpenSSL untuk mengamankan data yang dikirimkan melalui jaringan komputer, seperti dalam proses login, transaksi online, dan pengiriman email [9]. OpenSSL juga mendukung implementasi Public Key Infrastructure (PKI) dengan menyediakan fitur untuk membuat dan mengelola sertifikat digital X.509, Certificate Signing Request (CSR), serta Certificate Revocation List (CRL) [10]. Fitur ini digunakan untuk membangun sistem keamanan berbasis sertifikat digital yang umumnya diterapkan dalam layanan seperti HTTPS serta berbagai aplikasi web yang membutuhkan komunikasi yang aman dan terenkripsi.

c. Tanda Tangan Digital *(Digital Signature)*

Tanda tangan digital merupakan suatu solusi teknologi yang digunakan untuk menjamin keaslian suatu dokumen elektronik serta memiliki tujuan untuk memvalisadi dokumen yang akan digunakan. Tanda tangan digital memastikan otentisitas dan integritas dokumen elektronik sehingga dapat diterima secara hukum [11]. Penggunaan tanda tangan digital di Indonesia diatur dalam Undang-Undang Nomor 11 Tahun 2008 tentang Informasi dan Transaksi Elektronik (ITE) dan Peraturan Pemerintah Nomor 71 Tahun 2019. Selain itu, tanda tangan digital juga diatur dalam Pasal 1 angka 12 Undang-Undang Nomor 19 tahun 2016 “Tanda Tangan Elektronik adalah tanda tangan yang terdiri atas Informasi Elektronik yang dilekatkan, terasosiasi atau terkait dengan Informasi

Elektronik lainnya yang digunakan sebagai alat verifikasi dan autentikasi" [12].

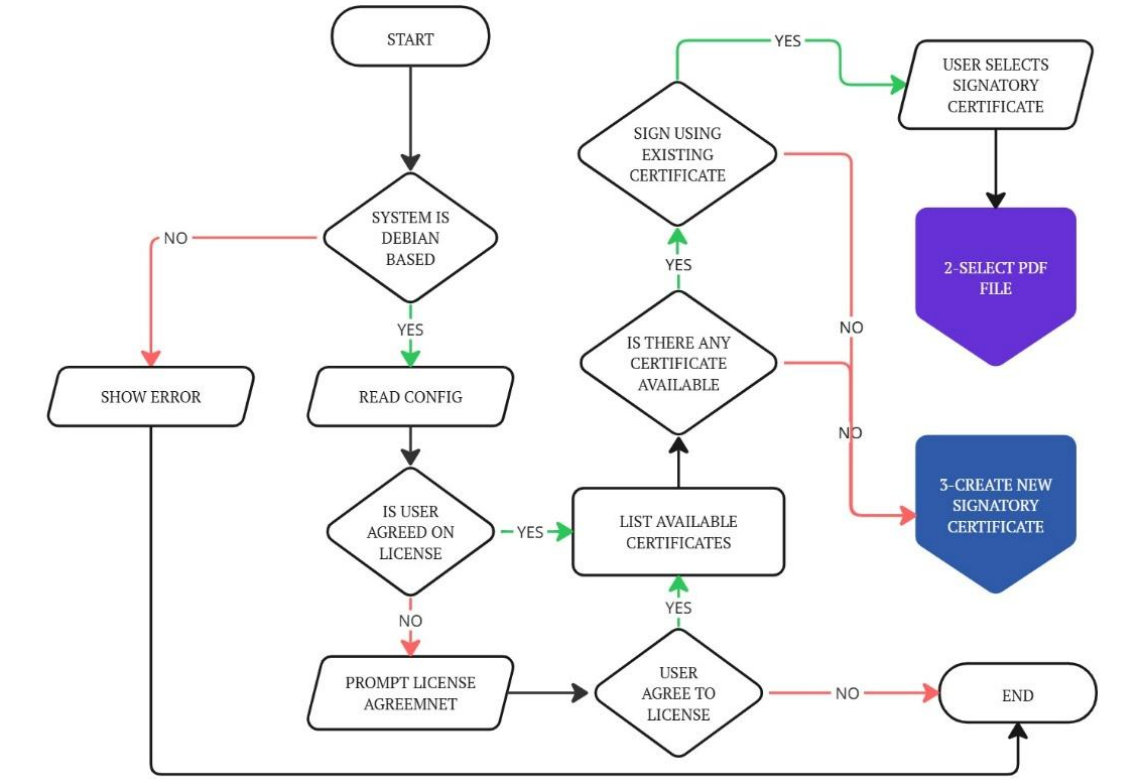
d. Kriptografi Asimetris

Kriptografi asimetris menggunakan sepasang kunci yang berbeda, yaitu kunci privat dan kunci publik yang digunakan untuk proses enkripsi dan juga deskripsi. Kunci privat digunakan untuk menandatangani dokumen, sedangkan kunci publik digunakan untuk memverifikasi tanda tangan tersebut [13]. Kriptografi asimetris berfungsi dengan dimulai dari pihak pengirim yang mengirimkan data dalam format *plaintext.* Langkah selanjutnya dalam kriptografi asimetris adalah melakukan proses enkripsi menggunakan kunci publik. Berkas yang telah dienkripsi akan berubah menjadi *ciphertext* dan pihak penerima akan mendeskripsi berkas *chiphertext* tersebut menggunakan kunci privat sehingga kembali menjadi format *plaintext* [14]. Salah satu algoritma yang umum digunakan dalam tanda tangan digital adalah RSA (Rivest-Shamir-Adleman).

RSA merupakan kriptografi asimetris yang didasarkan pada operasi eksponensial terbatas dalam bilangan bulat (ZN). RSA memiliki dua pasang algoritma, yaitu algoritma enkripsi publik dan algoritma deskripsi. Enkripsi dan deskripsi tersebut didasarkan pada konsep bilangan aritmetika dan aritmetika modulo. Kekuatan yang dimiliki oleh algoritma RSA ini adalah pada proses eksponensial dan pemfaktoran bilangan non-prima menjadi faktor prima untuk mendapatkan kunci deskripsi [15].

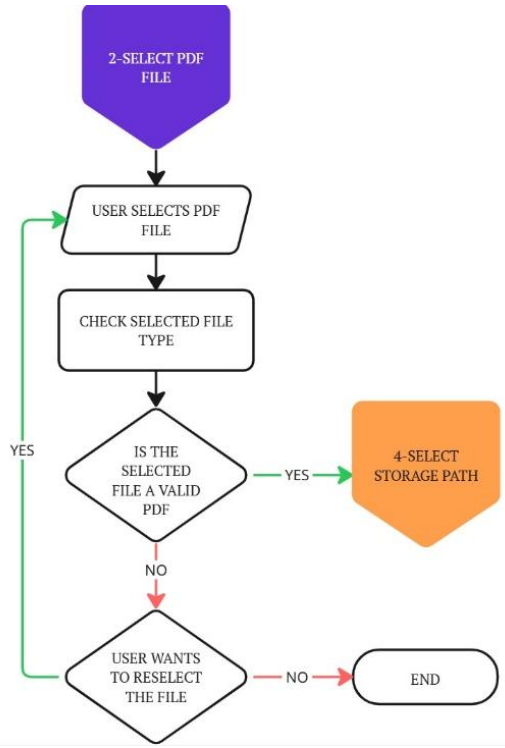
METODOLOGI

Skrip dimulai dengan memeriksa apakah dijalankan pada *operating system* (OS) yang berbasiskan Debian atau tidak. Jika tidak maka diberitahukan kepada pengguna bahwa skrip hanya bisa dijalankan pada sistem dengan OS berbasis debian kemudian skrip akan diakhiri. Selain itu, jika skrip dijalankan pada sistem dengan OS berbasis Debian maka skrip kemudian membaca file konfigurasi yang berada dalam direktori relatif ./resources/.config yang berisi data dengan format KEY=VALUE. Skrip kemudian memeriksa apakah pengguna telah atau pernah menyetujui persetujuan lisensi berdasarkan nilai dari *key* LICENSE_AGREED yang jika bernilai TRUE maka diasumsikan pengguna telah menyetujui persetujuan lisensi. Jika LICENSE_AGREED bernilai selain TRUE maka ditampilkan jendela yang menampilkan ketentuan lisensi yang terdapat pada direktori relatif ./resources/LICENSE.html. Jika pengguna setuju dengan ketentuan lisensi maka skrip dilanjutkan selain itu skrip diakhiri. Jika dilanjutkan maka skrip kemudian mendaftar semua sertifikat yang tersedia pada direktori relatif ./certs, jika tidak ada satupun sertifikat tersedia maka skrip langsung mengeksekusi algoritma untuk membuat sertifikat RSA baru guna menandatangani file. Jika terdapat minimal satu sertifikat pada direktori ./certs maka skrip mengeksekusi algoritma yang memungkinkan pengguna untuk memilih sertifikat yang digunakan untuk menandatangani dan file PDF (*Portable Document Format*) yang ingin ditandatangani secara elektronik menggunakan sertifikat yang dipilih. Tahapan tersebut divisualisasikan pada Gambar 1.



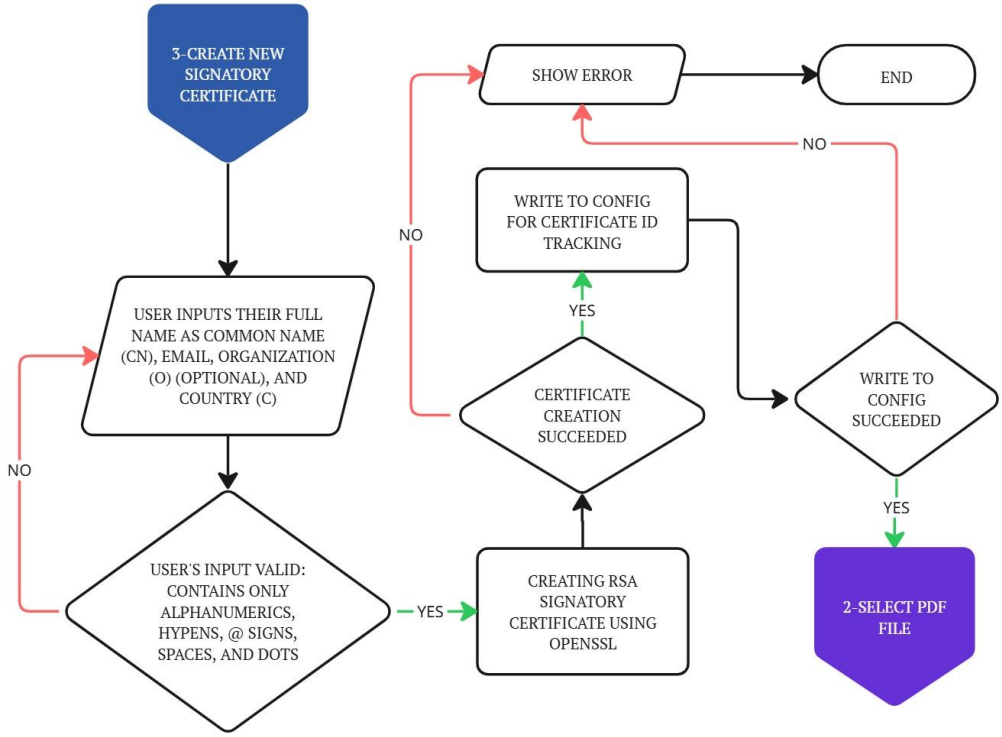
Gambar 1. *Flowchart* utama skrip

Algoritma pemilihan file PDF dimulai meminta pengguna untuk memilih file dengan ekstensi PDF lalu memeriksa apakah file yang dipilih adalah PDF yang valid dengan memeriksa tidak hanya ekstensi file namun juga *MIME type* file tersebut. Jika file yang dipilih adalah PDF yang valid maka skrip akan mengeksekusi algoritma untuk memilih lokasi penyimpanan file yang telah ditandatangani nantinya. Selain itu, jika file yang dipilih bukanlah file PDF yang valid maka ditampilkan pesan yang menginformasikan kepada pengguna bahwa file yang dipilih bukanlah PDF yang valid kemudian meminta pengguna untuk memutuskan apakah ingin memilih ulang file atau mengakhiri proses. Jika pengguna memutuskan untuk memilih ulang file maka skrip memulai kembali algoritma yang meminta pengguna untuk memilih file, selain itu skrip diakhiri. Algoritma tersebut divisualisasikan pada Gambar 2.



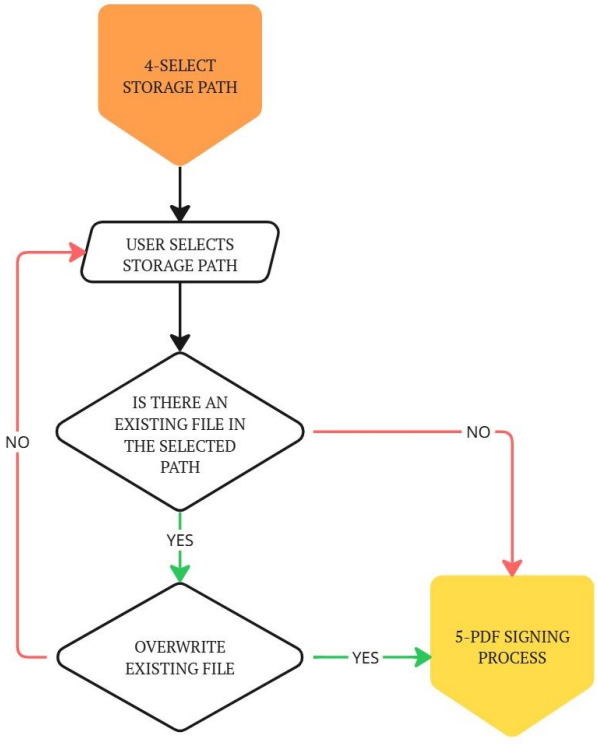
Gambar 2. *Flowchart* algoritma pemilihan file PDF

Algoritma pembuatan sertifikat RSA dimulai dengan meminta pengguna untuk memasukkan nama lengkap sebagai *Common Name* (CN), alamat email sebagai Email (E), nama organisasi tempat bekerja sebagai *Organization* (O) yang merupakan isian opsional yang jika dikosongkan maka secara otomatis akan diisi sama dengan isian CN oleh skrip, dan negara sebagai *Country* (C). Skrip kemudian melakukan sanitasi masukan (*input sanitization*) terhadap data yang telah dimasukkan oleh pengguna dengan memeriksa apakah semua masukan hanya berupa karakter-karakter alfanumerik, tanda hubung, simbol @, spasi, dan titik. Jika data dari pengguna tidak lolos sanitasi maka ditampilkan pesan kesalahan yang menginformasikan ke pengguna terkait kesalahan data yang dimasukkan, selain itu skrip akan dilanjutkan dengan pembuatan sertifikat RSA *self-signed* menggunakan *tool* bernama openssl berdasarkan data-data yang telah dimasukkan oleh pengguna. Jika pembuatan sertifikat baru berhasil maka dibaca nilai dari *key* LAST_CERT_ID yang ada pada file ./resources/.config yang berupa *integer* dan merupakan *incremental certificate ID* yang digunakan untuk menandai ID sertifikat terakhir yang dibuat. Nilai *incremental certificate ID* ditambahkan satu setiap pembuatan sertifikat RSA baru yang berhasil lalu menuliskan nilainya kembali ke *key* LAST_CERT_ID. Nilai ID yang telah ditambahkan satu kemudian dijadikan nama folder dalam direktori ./certs dengan format penamaan <ID>|<CN> lalu menyimpan pasangan kunci (*key pair*) RSA yang terdiri dari fullchain.pem dan privkey.pem yang baru dibuat ke dalam folder tersebut, kemudian skrip langsung mengeksekusi algoritma untuk memungkinkan pengguna memilih file PDF yang akan ditandatangani menggunakan sertifikat yang baru dibuat. Jika sertifikat RSA gagal dibuat maka ditampilkan informasi kesalahan kemudian skrip diakhiri. Algoritma pembuatan sertifikat RSA divisualisasikan pada Gambar 3.



Gambar 3. *Flowchart* pembuatan sertifikat RSA

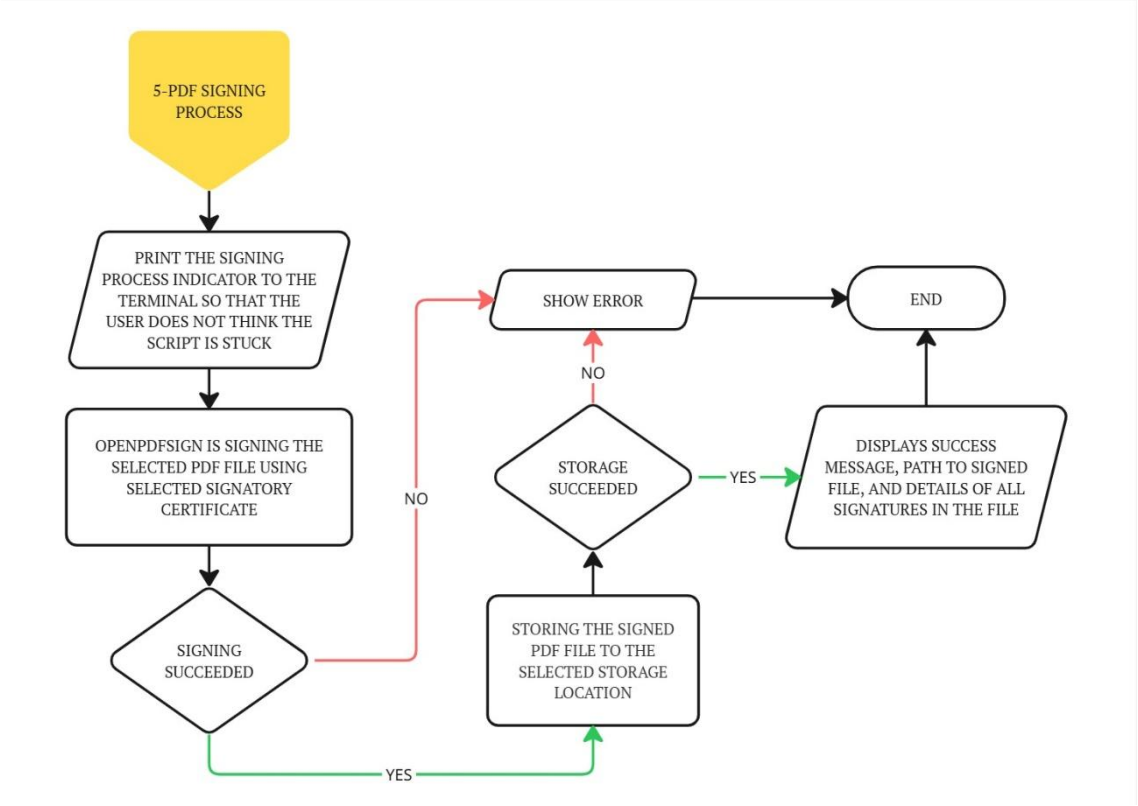
Algoritma pemilihan lokasi penyimpanan file yang telah ditandatangani dimulai dengan meminta pengguna untuk memilih direktori tempat file yang telah ditandatangani akan disimpan, kemudian memeriksa apakah direktori yang dipilih pengguna telah terdapat file. Jika pada direktori yang dipilih telah terdapat file maka ditampilkan jendela yang menginformasikan kepada pengguna bahwa direktori tersebut telah terdapat file dan meminta konfirmasi dari pengguna untuk menimpa file yang telah ada. Jika pengguna menolak menimpa file maka skrip akan meminta pengguna untuk memilih kembali direktori penyimpanan. Jika pengguna memilih untuk menimpa file yang telah ada maka skrip akan mengeksekusi algoritma untuk menandatangani file menggunakan sertifikat yang telah dipilih atau dibuat sebelumnya. Hal yang sama juga berlaku jika pengguna memilih lokasi penyimpanan yang tidak tumpang tindih dengan file yang telah ada maka skrip akan langsung mengeksekusi algoritma penandatanganan. Tahapan tersebut divisualisasikan pada Gambar 4.



Gambar 4. *Flowchart* pemilihan lokasi penyimpanan file yang ditandatangani

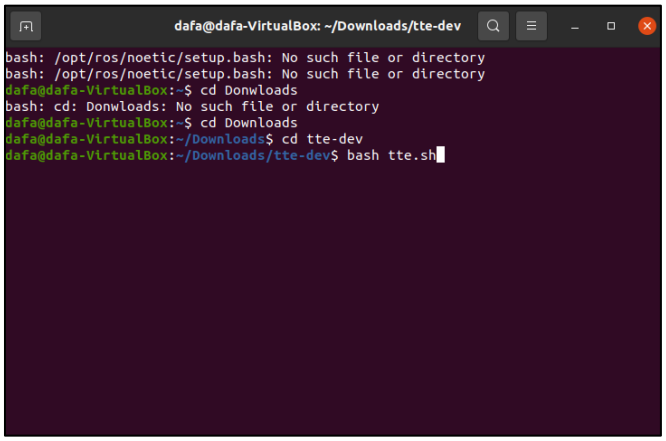
Algoritma penandatanganan dimulai dengan mencetak informasi kemajuan proses penandatangan ke dalam terminal

untuk meminimalisir pengguna mengira bahwa eksekusi skrip macet, kemudian dilakukan proses kriptografi berupa penandatanganan file PDF menggunakan *tool* bernama openpsfsign yang parameternya disesuaikan dengan masukan dari pengguna pada langkah-langkah sebelumnya. Jika proses penandatanganan file berhasil maka file yang telah ditandatangani disimpan pada direktori yang telah dipilih sebelumnya. Jika proses penandatanganan file gagal atau proses penyimpanan file yang telah ditandatangani tidak berhasil maka ditampilkan detail kesalahan kepada pengguna lalu skrip diakhiri. Selain itu jika proses penandatanganan file dan proses penyimpanan file berhasil maka ditampilkan pesan yang menginformasikan bahwa file telah berhasil ditandatangani dan berhasil disimpan pada direktori yang dipilih. Ditampilkan juga detail semua tanda tangan elektronik yang ada pada file tersebut kemudian eksekusi skrip diakhiri. Algoritma proses penandatanganan file digital menggunakan sertifikat RSA divisualisasikan pada Gambar 5.



Gambar 5. *Flowchart* proses penandatanganan file secara elektronik

HASIL DAN PEMBAHASAN

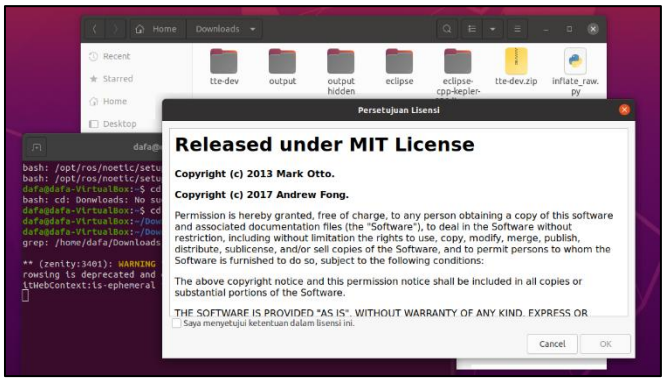


Gambar 6. Konfigurasi awal pada terminal ubuntu

Tahapan awal dalam proses eksekusi program ditunjukkan pada Gambar 6. Dimana program TTE dijalankan melalui terminal Ubuntu. Program ini dijalankan dengan menggunakan proses sebagai berikut (dengan asumsi file program tte terletak di local Download) :

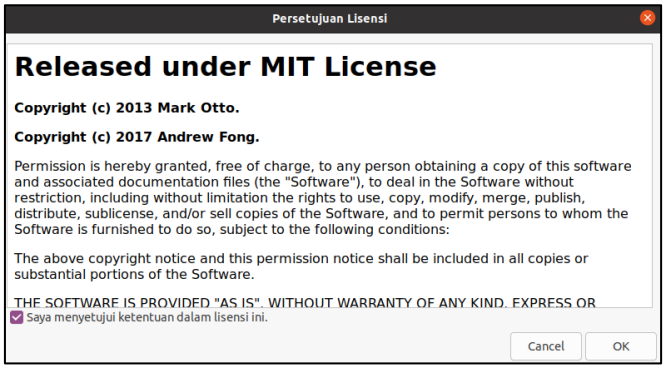
1. Buka terminal ubuntu
2. Ketik perintah ‘cd download’
3. jalankan program dengan perintah ‘bash tte.sh’

Setelah perintah bash tte.sh dijalankan, akan muncul tampilan seperti pada Gambar 7 yang dibawah ini.



Gambar 7. Lisensi Autentifikasi pengguna

Program akan secara otomatis membuka lisensi program yang menandakan bahwa ‘program baru pertama kali dijalankan’ dan pengguna harus menyetujui lisensi dari program tersebut untuk lanjut dalam penggunaan program. Jika pengguna tidak mencentang lisensi, artinya pengguna tidak menyetujui ketentuan yang berlaku dari program, maka program tidak bisa dijalankan dan pengguna hanya akan terhenti di terminal linux. Prosedur ini selaras dengan mekanisme validasi yang telah dijelaskan pada bagian metodologi, di mana nilai dari LICENSE_AGREED akan diperbarui jika pengguna menyetujui lisensi.



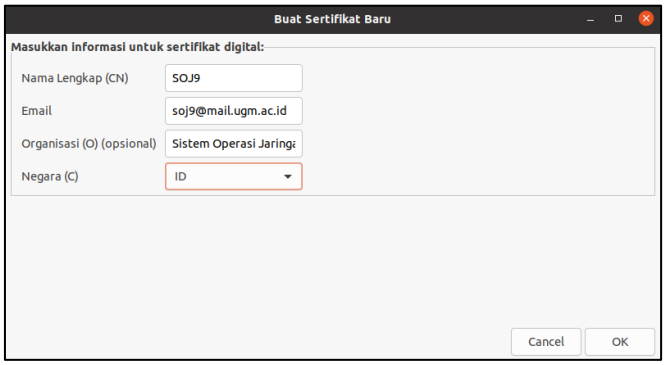
Gambar 8. Aktivasi lisensi

Kemudian klik centang ‘persetujuan lisensi’ yang muncul, agar config dari lisensinya aktif dan file dapat dijalankan.



Gambar 9. Opsi membuat sertifikat baru dan menggunakan sertifikat yang ada

Setelah lisensi diaktifkan, program melanjutkan ke tahap identifikasi sertifikat. Lalu program akan memicu pemilihan antara sertifikat RSA yang ada atau pembuatan sertifikat baru Jika belum tersedia sertifikat yang sesuai pada direktori, yang ditunjukkan oleh Gambar 9. Setelah memilih ‘OK’ untuk membuat sertifikat baru, GUI akan berubah menjadi seperti Gambar 10 di bawah ini.



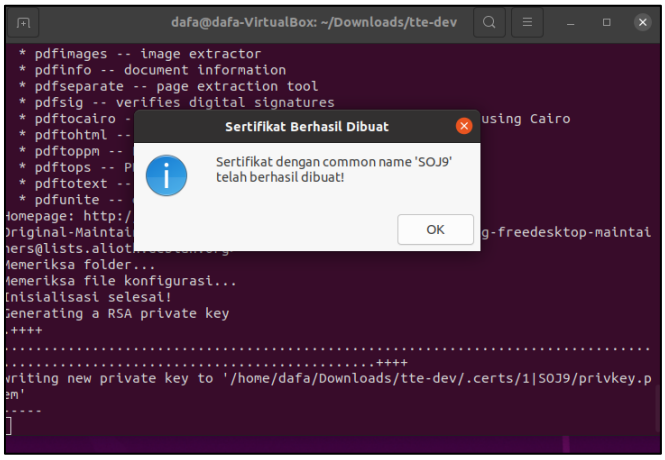
Gambar 10. Input data pembuatan sertifikat digital

Sekarang, pengguna diminta untuk mengisi data yang akan digunakan untuk membangun sertifikat digital,, misalnya:

1. Name Lengkap (CN): SOJ9

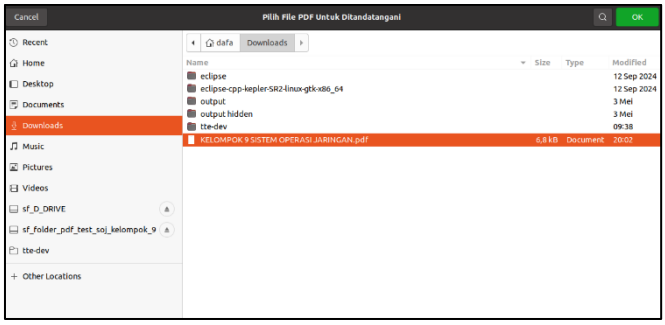
2. Email: soj9@mail.ugm.ac.id
3. Organisasi (O): Sistem Operasi Jaringan (opsional)
4. Negara (C): ID

Setelah input tervalidasi, program menggunakan tool openssl untuk membuat sertifikat RSA yang bersifat *self-signed*. Hasil dari proses ini akan disimpan di direktori ./certs dengan format penamaan sesuai ID dan CN, misalnya 01|SOJ9. Sebagai catatan tambahan, ketika kolom 'Organisasi (O)' tidak di isi, maka datanya akan tetap terisi otomatis sesuai data dari kolom 'Nama Lengkap'.



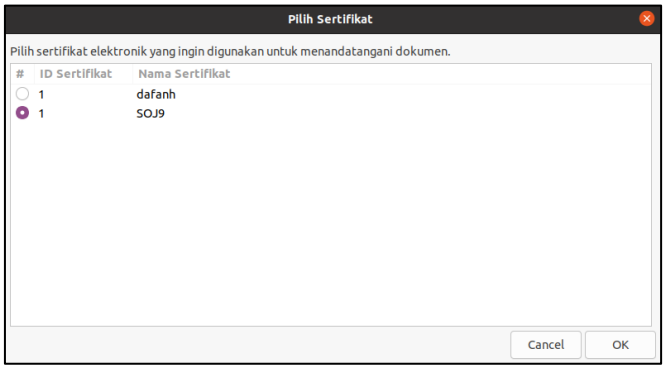
Gambar 11. Hasil *output* pembuatan sertifikat

Seperti yang dapat dilihat dari Gambar 11, sertifikat 'SOJ9' berhasil dibuat setelah program dijalankan. Setelah menekan tombol "OK", pengguna dapat melanjutkan untuk memilih file PDF yang akan digunakan sebagai media penandatanganan elektronik.



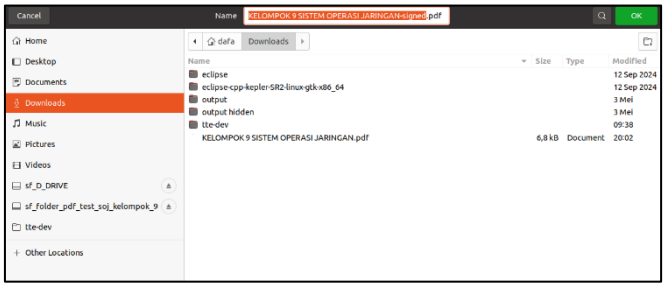
Gambar 12. Pemilihan file pdf yang akan ditandatangani

Tahapan selanjutnya, seperti yang dapat dilihat pada Gambar 12, adalah pemilihan file PDF yang akan ditandatangani secara elektronik. Yang yaitu suatu file PDF bernama "KELOMPOK 9 SISTEM OPERASI JARINGAN.pdf", sebagai contoh percobaan dari program 'tte' ini.



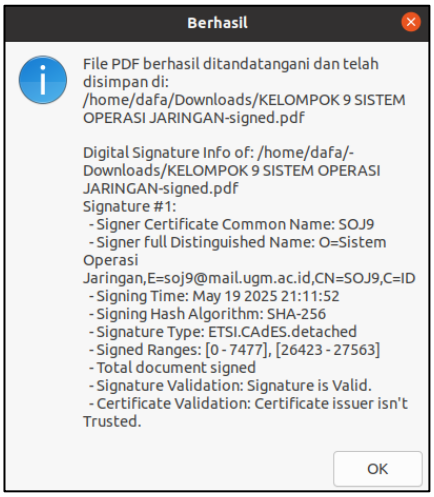
Gambar 13. Opsi pemilihan sertifikat yang dipakai

Setelah file dipilih, pengguna akan diminta untuk menentukan sertifikat yang akan digunakan untuk menandatangani file tersebut. Dalam kasus ini, dipilih sertifikat SOJ9 yang sebelumnya telah dibuat (Gambar 13).



Gambar 14. Penamaan file yang telah di beri tanda tangan elektronik

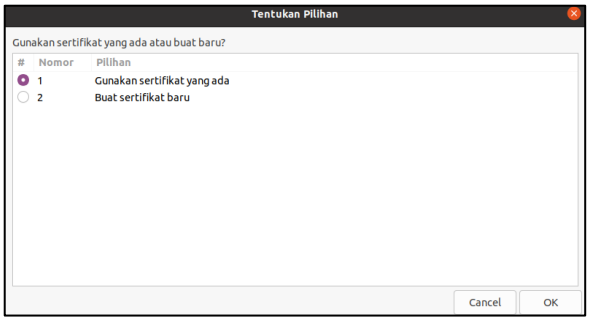
Setelah proses penandatanganan selesai, file hasil (KELOMPOK 9 SISTEM OPERASI JARINGAN-signed.pdf) berhasil disimpan dan dapat diverifikasi melalui metadata yang tertanam dalam PDF tersebut.



Gambar 15. Hasil notifikasi program berhasil dijalankan

Setelah proses penandatanganan berhasil, program menampilkan pop-up notifikasi yang menyatakan bahwa file telah berhasil ditandatangani. Pada notifikasi tersebut juga disampaikan bahwa terdapat satu file dengan ID SOJ9 yang telah ditandatangani secara elektronik. File hasil ditandatangani tersebut disimpan di direktori Downloads, sesuai dengan lokasi yang telah ditentukan pada saat pemberian nama file sebelumnya.

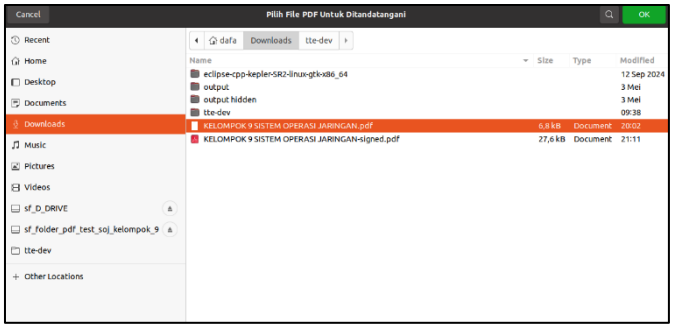
Setelah selesai untuk pengujian file tadi. Sekarang kita lanjut uji coba lagi program, dimana daripada membuat sertifikat baru, akan menggunakan file sertifikat yang telah dibuat tadi.



Gambar 16. Opsi membuat sertifikat baru dan menggunakan sertifikat

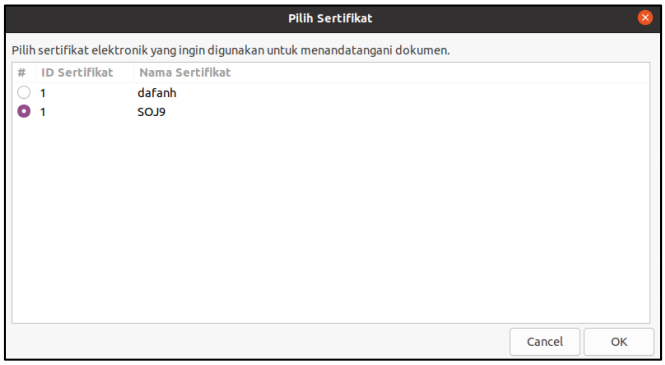
Setelah melakukan kembali langkah awal menjalankan programnya melalui bash tte.sh. Sekarang, pengguna langsung diminta untuk memilih opsi antara ‘Gunakan sertifikat yang ada’ dan ‘Buat sertifikat baru’.

Dengan perbedaan untuk pengulangan ini, sekarang memilih opsi 'Gunakan sertifikat yang ada’ membuktikan sertifikat yang telah dibuat tadi. Setelah memilih ‘Gunakan sertifikat yang ada’, maka otomatis akan menampilkan menu pemilihan file pdf yang akan ditandatangani, seperti pada Gambar 17 di bawah ini, dan Gambar 12 pada sebelumnya.



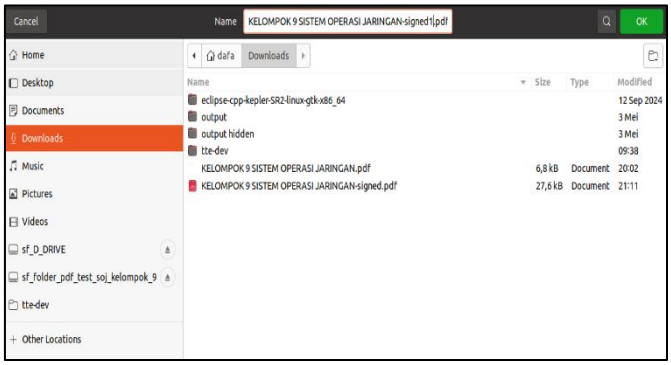
Gambar 17. Pemilihan file pdf yang akan ditandatangani

Tahapan selanjutnya, adalah pemilihan file PDF yang akan ditandatangani secara elektronik lagi. yaitu file PDF bernama “KELOMPOK 9 SISTEM OPERASI JARINGAN.pdf”, sebagai contoh percobaan dari program ‘tte’ ini.



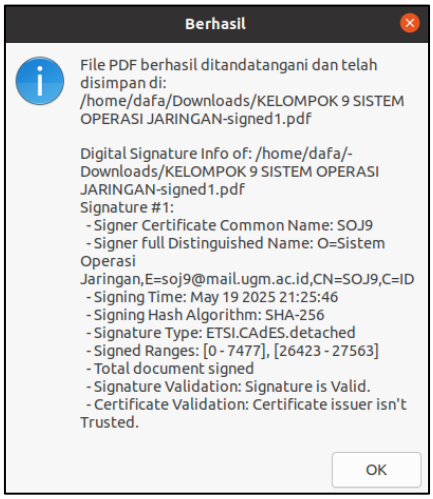
Gambar 18. Opsi pemilihan sertifikat yang dipakai

Setelah file dipilih, pengguna akan diminta untuk menentukan sertifikat yang akan digunakan untuk menandatangani file tersebut. Dalam kasus ini, dipilih sertifikat SOJ9 yang sebelumnya telah dibuat (Gambar 18).



Gambar 19. Penamaan file hasil tanda tangan elektronik

Setelah proses penandatanganan selesai, file hasil (KELOMPOK 9 SISTEM OPERASI JARINGAN-.signed1.pdf) berhasil disimpan dan dapat diverifikasi melalui metadata yang tertanam dalam PDF tersebut.

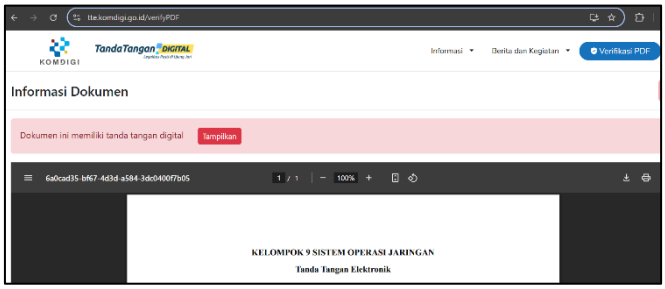


Gambar 20. Hasil notifikasi program berhasil dijalankan

Setelah proses penandatanganan berhasil, program menampilkan pop-up notifikasi yang menyatakan bahwa file telah berhasil ditandatangani. Pada notifikasi tersebut juga disampaikan bahwa terdapat satu file dengan ID SOJ9 yang telah ditandatangani secara elektronik. File hasil ditandatangani tersebut disimpan lagi seperti percobaan sebelumnya, yaitu di direktori Downloads, sesuai dengan lokasi yang telah ditentukan pada saat pemberian nama file sebelumnya.

Dengan demikian, program berhasil menjalankan keseluruhan alur program dari pembuatan sertifikat hingga penandatanganan digital file PDF. Maka dapat disimpulkan bahwa, pada program terdapat dua opsi yang tersedia, yakni ‘membuat baru sertifikat elektronik’ dan ‘Gunakan sertifikat yang telah ada' sebagai opsi untuk membuat sebuah sertifikat tanda tangan elektroniknya.

Sebagai langkah validasi akhir, dilakukan pengujian terhadap hasil tanda tangan digital menggunakan layanan verifikasi milik Direktorat Jenderal Aplikasi Informatika, Kementerian Komunikasi dan Informatika (Kominfo), yaitu melalui situs https://tte.komdigi.go.id/. File PDF yang telah ditandatangani menggunakan program tte.sh diunggah ke situs tersebut untuk mengonfirmasi keberhasilan proses penandatanganan digital.



Gambar 21. Hasil pengecekan tanda tangan digital melalui situs TTE Kominfo

Berdasarkan hasil verifikasi yang ditampilkan pada Gambar 21, sistem pada situs TTE Kominfo menampilkan notifikasi bahwa *"dokumen ini memiliki tanda tangan digital"*. Hal ini menandakan bahwa file telah berhasil ditandatangani secara elektronik. Selain itu, situs Kominfo tidak mengizinkan proses penandatanganan ulang pada file tersebut. Hal ini selaras dengan aturan bahwa file PDF hanya dapat diberi tanda tangan digital sekali, kecuali menggunakan fitur penandatanganan multipihak yang berbeda.

Hasil ini menunjukkan bahwa program tte.sh mampu menghasilkan tanda tangan elektronik yang sah dan dikenali oleh sistem verifikasi resmi pemerintah. Dengan demikian, dapat disimpulkan bahwa mekanisme pembuatan dan penyisipan tanda tangan digital menggunakan skrip bash dan sertifikat RSA yang dihasilkan telah berfungsi secara efektif dan sesuai standar TTE nasional.

Berdasarkan hasil pengujian yang telah dilakukan, dapat disimpulkan bahwa program *tte.sh* berhasil menjalankan seluruh tahapan proses tanda tangan digital secara fungsional dan sistematis. Dimulai dari proses eksekusi awal melalui terminal Ubuntu, pengguna diarahkan melalui alur konfigurasi yang mencakup persetujuan lisensi, pembuatan atau pemilihan sertifikat digital, pemilihan file PDF, hingga proses penandatanganan dan penyimpanan file yang telah ditandatangani. Tampilan antarmuka grafis yang terintegrasi dengan skrip bash memungkinkan interaksi pengguna yang lebih intuitif, meskipun program dijalankan dalam lingkungan terminal.

Prosedur pembuatan sertifikat menggunakan OpenSSL menunjukkan bahwa program tidak hanya terbatas pada fungsi tanda tangan, tetapi juga mampu menghasilkan sertifikat RSA *self-signed* yang disesuaikan dengan informasi pengguna. Hal ini penting mengingat validitas dari tanda tangan digital sangat bergantung pada kredensial dan keaslian dari sertifikat yang digunakan. Penempatan hasil sertifikat pada direktori khusus serta penggunaan format penamaan

yang sistematis menambah kejelasan dan keteraturan dalam manajemen sertifikat digital.

Keberhasilan program dalam menyisipkan tanda tangan digital pada file PDF dibuktikan melalui dua skenario pengujian, yaitu pembuatan sertifikat baru dan penggunaan kembali sertifikat yang telah dibuat. Dalam kedua skenario tersebut, program mampu mengidentifikasi sertifikat yang digunakan dan menyisipkan tanda tangan digital tanpa kesalahan. Hasil penandatanganan disimpan secara otomatis pada direktori yang ditentukan, menunjukkan bahwa proses output juga berjalan sesuai harapan. Notifikasi akhir yang muncul memberikan umpan balik yang jelas kepada pengguna terkait status dan hasil proses.

Langkah verifikasi akhir melalui situs resmi Tanda Tangan Elektronik Kominfo menjadi validasi krusial terhadap keabsahan tanda tangan digital yang dihasilkan. Situs TTE Kominfo secara otomatis mengenali bahwa file telah memiliki tanda tangan elektronik yang sah, dan menolak untuk melakukan penandatanganan ulang. Ini memperkuat kesimpulan bahwa format tanda tangan dan sertifikat yang dibangun oleh program *tte.sh* telah sesuai dengan standar nasional yang berlaku di Indonesia.

Dari keseluruhan pengujian, dapat dinyatakan bahwa program ini telah memenuhi fungsionalitas dasar dari sistem tanda tangan elektronik: autentikasi, integritas, dan non-repudiasi. Dengan penggunaan sertifikat berbasis RSA serta integrasi dengan antarmuka pengguna berbasis GUI dalam lingkungan bash, program ini membuktikan bahwa implementasi sistem tanda tangan elektronik tidak selalu memerlukan platform kompleks, melainkan dapat dilakukan secara efektif dengan pendekatan skrip dan tool open source yang tersedia secara umum.

Secara keseluruhan, program *tte.sh* menunjukkan potensi untuk diadopsi dalam skala lebih luas, baik untuk keperluan akademik maupun institusional, selama kebutuhan akan tanda tangan digital yang sah, aman, dan terverifikasi tetap menjadi prioritas. Untuk pengembangan lebih lanjut, fitur tambahan seperti penandatanganan multipihak, integrasi timestamp, atau dukungan untuk jenis sertifikat eksternal dapat menjadi arah pengembangan berikutnya.

Maka dari hasil dan pembahsan diatas, kami memakai materi yang ada di NetAcad, meliputi:

A. Bab 2-Sistem Operasi

- Memeriksa apakah skrip dijalankan dalam sistem berbasis Debian atau tidak.

B. Bab 3 Bekerja di Linux

- Memanfaatkan alat bantu manajemen paket dan repositori dengan menggunakan perintah apt-get untuk menginstal paket-paket yang dibutuhkan.

C. Modul 4 Perangkat Lunak Sumber Terbuka dan Lisensi

- Menggunakan lisensi sumber terbuka Lisensi MIT.

D. Modul 5-Keterampilan Baris Perintah

- Variabel
- Sintaks-sintaks baris perintah.
- Mengutip teks.

E. Modul 7- Menavigasi Sistem Berkas

- Melakukan CD-ing direktori kerja ke tempat skrip berada.
- Memanfaatkan file dan direktori tersembunyi untuk sumber daya dan direktori sertifikat.
- Memanfaatkan jalur absolut dan relatif.

F. Modul 8-Mengelola File dan Direktori

- Membuat file dan direktori untuk menyimpan konfigurasi dan sertifikat penandatangan dan juga PDF yang ditandatangani.
- Globbing.
- Memanfaatkan Sensitivitas Huruf Besar/Kecil dalam penamaan direktori yang digunakan untuk menyimpan sertifikat penandatangan.

G. Modul 10-Bekerja dengan Teks

- Membaca kunci tertentu dari file konfigurasi dengan menggunakan ekspresi reguler dasar RegEx.
- Memanfaatkan pipa baris perintah (1).
- Pengalihan I/O (> dan >>).

H. Modul 11-Skrip Dasar

- Memanfaatkan skrip bash dasar.

I. Bab 17-Kepemilikan dan Perijinan

- Memanfaatkan hak akses berkas dan direktori dengan membuat skrip bash dapat dieksekusi dengan memanggil chmod +x tte.sh.

SIMPULAN

Berdasarkan hasil studi literatur, analisis kebutuhan pengguna, hingga tahap implementasi dan pengujian yang telah dilakukan dari proyek "Pemanfaatan Bash Script untuk

Penandatanganan Dokumen Digital di Sistem Operasi Linux", maka dapat disimpulkan beberapa poin penting berikut ini:

1. Permasalahan yang Diidentifikasi

Permasalahan utama yang diangkat dalam penelitian ini adalah kurangnya sarana yang efisien, ringan, dan dapat diandalkan untuk melakukan penandatanganan dokumen digital pada lingkungan sistem operasi berbasis Linux, khususnya Debian. Di tengah kebutuhan yang meningkat terhadap keamanan dan keabsahan dokumen digital—yang secara hukum telah diatur dalam UU ITE dan PP No. 71 Tahun 2019—masih banyak pengguna yang menghadapi kendala teknis dalam menghasilkan dan menerapkan tanda tangan digital yang sah.

Permasalahan tersebut diperparah dengan keterbatasan akses terhadap perangkat lunak komersial, kompleksitas prosedur pembuatan sertifikat digital, dan ketiadaan antarmuka yang ramah pengguna di platform open-source. Selain itu, belum adanya sistem otomatisasi berbasis terminal yang mampu:

- Melakukan proses verifikasi sistem, persetujuan lisensi, hingga penyisipan tanda tangan digital pada dokumen.
- Menjamin validitas sertifikat digital yang dihasilkan.
- Menghindari kesalahan penggunaan, pemalsuan dokumen, atau manipulasi data PDF yang ditandatangani.

2. Solusi yang Ditawarkan

Sebagai solusi, penelitian ini mengembangkan sebuah skrip bash otomatisasi (bash scripting automation) yang dirancang untuk melakukan proses penandatanganan digital file PDF berbasis sistem operasi Linux Debian dengan pendekatan yang modular, sistematis, dan terintegrasi. Solusi ini menawarkan sejumlah keunggulan, antara lain:

- Verifikasi OS berbasis Debian sebelum skrip dijalankan.
- Validasi lisensi pengguna melalui pembacaan file konfigurasi .config dan file lisensi HTML (LICENSE.html).
- Pembuatan sertifikat RSA self-signed melalui tool OpenSSL, dengan masukan data Common Name (CN), Email, Organisasi (O), dan Negara (C), disertai sanitasi input.
- Pemilihan file PDF dengan validasi ekstensi dan MIME type, guna mencegah kesalahan input file non-valid.
- Pemilihan lokasi penyimpanan hasil file yang telah ditandatangani, termasuk opsi untuk menimpa atau memilih ulang direktori.
- Penandatanganan PDF menggunakan tool OpenPSFSign, dengan parameter sesuai data pengguna dan penyimpanan hasil di direktori tujuan.

3. Rincian Keluaran Sistem dan Manfaat

Sistem yang dibangun menghasilkan berbagai keluaran dan manfaat, yaitu:

a. Keluaran Sistem

- Sertifikat digital RSA self-signed tersimpan dalam direktori ./certs/ dengan penamaan sistematis <ID>|<CN>, yang berisi file fullchain.pem dan privkey.pem.
- File PDF yang telah ditandatangani secara digital dengan ekstensi .signed.pdf atau .signed1.pdf, disimpan di direktori pilihan pengguna.
- Log dan notifikasi status proses pada antarmuka terminal, mencakup validasi file, pemilihan sertifikat, hasil penandatanganan, dan informasi kesalahan jika ada.
- Notifikasi keberhasilan penandatanganan disertai informasi sertifikat yang digunakan.

b. Manfaat Sistem

- Kemudahan penggunaan bagi pengguna awam di Linux melalui antarmuka terminal yang terarah.
- Efisiensi waktu dan proses, karena seluruh tahapan dilakukan secara otomatis dan terintegrasi.
- Peningkatan keamanan dan keabsahan dokumen, berkat penggunaan sertifikat RSA dan validasi input ketat.
- Kompatibilitas dengan sistem verifikasi nasional, terbukti dari pengujian melalui situs resmi TTE Kominfo yang menyatakan bahwa file yang telah ditandatangani tidak dapat ditandatangani ulang karena sudah memiliki tanda tangan digital sah.

4. Implikasi Umum dari Penelitian dan Pengembangan Sistem

Penelitian ini menunjukkan bahwa solusi tanda tangan digital yang sah dan sesuai standar dapat dibangun hanya dengan memanfaatkan teknologi open-source seperti Bash, OpenSSL, dan OpenPSFSign, tanpa memerlukan perangkat lunak komersial atau kompleksitas antarmuka grafis.

Implikasi dari pengembangan sistem ini mencakup:

- Dukungan terhadap digitalisasi dokumen di instansi pendidikan, pemerintahan, maupun industri, terutama yang mengandalkan sistem Linux.
- Kontribusi terhadap prinsip transparansi dan akuntabilitas digital, yang sejalan dengan tujuan Sustainable Development Goals (SDG) nomor 16, yaitu membangun institusi yang efektif, bertanggung jawab, dan transparan.
- Peningkatan adopsi tanda tangan elektronik di lingkungan open-source, termasuk pada organisasi non-profit, komunitas pengembang, dan lembaga akademik.
- Peluang pengembangan fitur lanjutan, seperti multi-user signature, integrasi timestamp server, atau penggunaan sertifikat pihak ketiga yang diakui secara internasional.

5. Penutup

Secara keseluruhan, sistem tte.sh yang dikembangkan dalam penelitian ini berhasil memberikan solusi praktis, legal, dan efisien terhadap kebutuhan penandatanganan dokumen digital berbasis Linux. Sistem ini terbukti fungsional dan sesuai standar nasional, serta memberikan pengalaman pengguna yang mudah diakses melalui terminal.

Dengan pendekatan berbasis skrip dan tool open-source, sistem ini mampu menjembatani kebutuhan keamanan digital dengan implementasi teknis yang ringan dan dapat digunakan secara luas. Diharapkan, hasil penelitian ini menjadi langkah awal menuju digitalisasi otentikasi dokumen yang terdesentralisasi dan terjangkau, serta menjadi dasar untuk pengembangan lebih lanjut yang berskala nasional maupun global.

REFERENSI

[1] Pratama Jr, M. D. Y. A., Arnaldy, D., TP, S., & Si, M. Analisis Integritas Dokumen Digital Pada Aplikasi Digisign UTD PNJ Menggunakan Tanda Tangan Digital.

[2] Amaludin, L., & Rahmatulloh, A. (2024). Penerapan ECDSA dan BLAKE2B Untuk Membentuk Tanda Tangan Digital Sebagai Autentikasi Dokumen. *Jurnal Informatika dan Multimedia*, *16*(2), 20-26.

[3] A. Petcu, M. Frunzete and D. A. Stoichescu, "A Practical Implementation Of A Digital Document Signature System Using Blockchain," 2023 13th International Symposium on Advanced Topics in Electrical Engineering (ATEE), Bucharest, Romania, 2023, pp. 1-6, doi: 10.1109/ATEE58038.2023.10108308.

[4] D. Dal and E. Çelik, "Evaluation of the Predictability of Passwords of Computer Engineering Students," *2022 3rd International Informatics and Software Engineering Conference (IISEC)*, Ankara, Turkey, 2022, pp. 1-6, doi: 10.1109/IISEC56263.2022.9998292.

[5] Hope Sr, K. R. (2020). Peace, justice and inclusive institutions: overcoming challenges to the implementation of Sustainable Development Goal 16. *Global Change, Peace & Security*, *32*(1), 57-77.

[6] Slameto, A. A., & Lukman, L. (2015). PENERAPAN OPENSSH DAN BASH SCRIPT UNTUK SIMULTANEOUS REMOTE ACCESS CLIENT PADA LABORATORIUM STMIK AMIKOM YOGYAKARTA. *SEMNASTEKNOMEDIA ONLINE*, *3*(1), 1-2.

[7] P. Feng, N. Xi, J. Jin, J. Zhang, dan J. Ma, "Flash: Deteksi Skrip Bash Berbahaya Berbasis Pembelajaran Grafik Terfederasi untuk Sistem Siber–Fisik Industri," dalam *IEEE Transactions on Industrial Informatics* , vol. 21, no. 6, hlm. 4979-4989, Juni 2025, doi: 10.1109/TII.2025.3552649.

[8] Goudarzi, D., & Valadon, G. Exploring OpenSSL Engines to Smash Cryptography.

[9] D. D. Kumar, J. D. Mukharzee, C. V. D. Reddy and S. M. Rajagopal, "Safe and Secure Communication Using SSL/TLS," *2024 International Conference on Emerging Smart Computing and Informatics (ESCI)*, Pune, India, 2024, pp. 1-6, doi: 10.1109/ESCI59607.2024.10497224.

[10] Husaini, H., Ramadhan, T. H., & Ihsan, M. (2025). PENERAPAN HIERARKI CERTIFICATE AUTHORITY DAN PUBLIC KEY INFRASTRUCTURE UNTUK MEMPERKUAT KEAMANAN JARINGAN. *Cyberspace: Jurnal Pendidikan Teknologi Informasi*, *9*(1), 63-74.

[11] Sulaiman, E., Arifudin, N., & Triyana, L. (2021). Kekuatan Hukum Digital Signature Sebagai Alat Bukti Yang Sah Di Tinjau Dari Hukum Acara Perdata. Risalah Hukum, 16, 95–105. https://doi.org/10.30872/risalah.v16i2.207

[12] F. B. Muzakkir, H. A. Darwito and M. Yuliana, "Developing Web-Based Application for QR Code Digital Signatures using OpenSSL," *2024 International Electronics Symposium (IES)*, Denpasar, Indonesia, 2024, pp. 386-392, doi: 10.1109/IES63037.2024.10665883.

[13] Basri, B. (2016). Kriptografi Simetris dan Asimetris dalam Perspektif Keamanan Data dan Kompleksitas Komputasi. *Jurnal Ilmiah Ilmu Komputer*, *2*(2), 16-23.

[14] Putra, N. B. N., Raihana, F. A., Mondong, W. M. A., & Kardian, A. R. (2023). Analisis Enkripsi Kriptografi Asimetris Algoritma RSA Berbasis Pemrograman Batch pada Media Flashdisk. *Jurasik (Jurnal Riset Sistem Informasi dan Teknik Informatika)*, *8*(1), 142-154.